

# 03 — Two Pointers & Sliding Window

DSA in Python — Interview Prep

---

## Two Pointers & Sliding Window — Tutorial

Covers the 10 problems solved in this folder. Where topic 01 used two pointers mostly to \*mirror\* (swap ends inward) and topic 02 used hashing to trade space for  $O(1)$  lookups, this topic is about two pointers (or a pointer pair defining a window) moving \*independently\* based on a running condition — sum, character count, or comparison result.

### The patterns

#### 1. Fixed-size window (01, 05)

Window length is given up front and never changes. Compute the window's state once, then slide by exactly one step at a time: add what enters on the right, remove what leaves on the left.

- `max_average_subarray`: state = running sum.
- `permutation_in_string`: state = a character-count dict; slide the same way, comparing the whole dict to a target dict each step.

Avoids recomputing from scratch at every position — the difference between  $O(n)$  and  $O(n \cdot k)$ .

#### 2. Variable-size window, grow-then-shrink (02, 03, 04)

Window length isn't fixed; `right` always advances, `left` advances conditionally based on whether the current window is still valid. Two flavors depending on what "valid" means:

- **Shrink while invalid** (`longest_substring_without_repeating`, `longest_repeating_character_replacement`): keep growing; the moment the window breaks its constraint, shrink from the left until it's valid again, then keep growing. Answer = best window length seen at any point.
- **Shrink while (still) valid, to find the minimum** (`minimum_size_subarray_sum`): grow until the window satisfies the condition, then shrink as far as possible \*while it still does\*, recording the shortest length at each shrink step, before growing again.

The tell for "this is a variable window problem": the question asks for the longest/shortest substring or subarray satisfying some running condition (sum, distinct-character count, replacement budget) — as opposed to "does one exist" (that's often plain hashing, topic 02) or "count all of them" (often prefix-sum + hash, also topic 02).

#### 3. Converging two-pointer on sorted data (06, 07)

Only works because the data is sorted: comparing the current pair's sum/value to a target tells you \*unambiguously\* which side to move.

- `two_sum_sorted`: sum too small -> move `left` right (only way to increase). Sum too big -> move `right` left.

- `three_sum`: fix one element via a loop, then run the exact same two-pointer routine on the rest for "sum to `-nums[i]`". Sorting also makes duplicate-skipping nearly free, since equal values become adjacent.

This is strictly better than hashing's `two_sum` (topic 02, 01) \*when the array is already sorted\* —  $O(1)$  space instead of  $O(n)$ , since the greedy pointer movement replaces the need to remember what was seen.

#### 4. Converging two-pointer via greedy elimination (08)

`container_with_most_water` isn't sorted, so it's not the same justification as pattern 3 — instead, the area is capped by the \*shorter\* of the two current lines, so moving the taller side's pointer inward can never help (still capped by the same shorter side, less width). Moving the shorter side is the only move that could possibly improve things. That greedy argument — one specific move is provably never better — is what justifies collapsing an  $O(n^2)$  all-pairs check into  $O(n)$ .

#### 5. Three-pointer partition (09)

`sort_colors` generalizes the read/write pointer from topic 01 to three regions instead of two: `low/mid` boundary for one value, `mid/high` boundary for another, with `mid` itself doing the classifying. The asymmetry — advancing `mid` after a 0-swap but not after a 2-swap — comes from \*what's known\* about the swapped-in value at each boundary (see the comment in `09_sort_colors.py` for the full argument). This pattern generalizes to "partition into  $k$  labeled groups in one pass."

#### 6. Fast-slow pointers / cycle detection preview (10)

`find_the_duplicate_number` treats an array as an implicit linked list (`i -> nums[i]`) and applies Floyd's cycle detection: a fast pointer moving 2 steps and a slow pointer moving 1 step will meet inside a cycle if one exists; resetting one pointer to the start and advancing both by 1 step finds the cycle's entrance. This is included here specifically as a preview — the exact same two-phase algorithm returns in `04_linked_list` for real cycle-detection problems, just with `.next` instead of `nums[i]`.

### Complexity summary

| #  | Problem                                 | Time     | Extra space                   |
|----|-----------------------------------------|----------|-------------------------------|
| 01 | max_average_subarray                    | $O(n)$   | $O(1)$                        |
| 02 | longest_substring_without_repeating     | $O(n)$   | $O(\min(n, \text{alphabet}))$ |
| 03 | minimum_size_subarray_sum               | $O(n)$   | $O(1)$                        |
| 04 | longest_repeating_character_replacement | $O(n)$   | $O(\text{alphabet})$          |
| 05 | permutation_in_string                   | $O(n)$   | $O(\text{alphabet})$          |
| 06 | two_sum_sorted                          | $O(n)$   | $O(1)$                        |
| 07 | three_sum                               | $O(n^2)$ | $O(1)$ (excl. sort/output)    |
| 08 | container_with_most_water               | $O(n)$   | $O(1)$                        |
| 09 | sort_colors                             | $O(n)$   | $O(1)$                        |
| 10 | find_the_duplicate_number               | $O(n)$   | $O(1)$                        |

Note 03 and 04's inner `while/shrink` loop looks like it could be  $O(n^2)$  nested, but `left` only ever moves forward and never resets — across the whole run it advances at most  $n$  times total, so the true cost is  $O(n)$  amortized,

not  $O(n)$  per outer step. Worth being able to explain this amortized-analysis argument out loud in an interview.

## Common interview follow-ups to be ready for

- `longest_substring_without_repeating`: "what if the alphabet is huge (unicode)?" — dict-based `last_seen` already handles this; a fixed-size array trick (common for ASCII-only variants) would not.
- `minimum_size_subarray_sum`: "what if nums can contain negative numbers?" — the shrink logic breaks (sum isn't monotonic as the window grows), which is exactly why `07_subarray_sum_equals_k` in topic 02 needed prefix-sum + hash instead of a sliding window.
- `three_sum`: "what about 4Sum / kSum?" — generalizes by fixing  $k-2$  elements with nested loops and running the same two-pointer on the remainder; same duplicate-skipping logic applies at each fixed level.
- `container_with_most_water`: be ready to justify out loud *\*why\** moving the taller pointer can never be optimal — interviewers often push specifically on this proof step, not just the code.
- `sort_colors`: "what if there were  $k$  colors instead of 3?" — the three-pointer trick doesn't generalize cleanly past 3; a counting sort (frequency count + overwrite) becomes the right answer instead.
- `find_the_duplicate_number`: "what if you could modify the array or use  $O(n)$  space?" — then it degenerates to `02_hashing/02_contains_duplicate`'s set-based approach; know why the *\* $O(1)$ -space, can't-modify\** constraints specifically force Floyd's algorithm here.

## Self-check before moving on

You should be able to, without looking at the code:

- State the difference between "shrink while invalid" and "shrink while valid" sliding windows, and give one example of each.
- Explain the amortized  $O(n)$  argument for why `left` never causing nested-loop blowup in `03/04`.
- Justify on paper why `container_with_most_water`'s greedy pointer move is safe.
- Walk through Floyd's cycle detection on a small example by hand (both phases) for `find_the_duplicate_number`.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `04_linked_list`.

# Programs

## 01. Max Average Subarray

01\_max\_average\_subarray.py

### Explanation

Problem: Given an integer array `nums` and integer `k`, find the maximum average value of any contiguous subarray of length `k`.

Source : LeetCode 643 - Maximum Average Subarray I

Example:

Input: `nums = [1, 12, -5, -6, 50, 3]`, `k = 4`

Output: 12.75 (subarray `[12, -5, -6, 50]`, sum `51 / 4`)

Idea: fixed-size sliding window, the simplest window shape. Compute the sum of the first `k` elements once, then slide the window one step at a time: add the element entering on the right, subtract the one leaving on the left. Avoids recomputing the sum from scratch at every position (which would be  $O(n*k)$ ).

### Code

```
def find_max_average(nums: list[int], k: int) -> float:
    window_sum = sum(nums[:k])
    max_sum = window_sum

    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        max_sum = max(max_sum, window_sum)

    return max_sum / k

if __name__ == "__main__":
    tests = [
        ([1, 12, -5, -6, 50, 3], 4, 12.75),
        ([5], 1, 5.0),
        ([-1], 1, -1.0),
        ([0, 1, 1, 3, 3], 4, 2.0),
    ]

    for i, (nums, k, expected) in enumerate(tests, 1):
        got = find_max_average(nums, k)
        status = "PASS" if abs(got - expected) < 1e-9 else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 02. Longest Substring Without Repeating

02\_longest\_substring\_without\_repeating.py

### Explanation

Problem: Given a string s, find the length of the longest substring without repeating characters.

Source : LeetCode 3 - Longest Substring Without Repeating Characters

Example:

Input: "abcabcbb"

Output: 3 ("abc")

Idea: variable-size sliding window. **right** extends the window one character at a time. A dict tracks the last index each character was seen at. If the character at **right** was seen before \*and\* that occurrence is still inside the current window, jump **left** to just past it - no need to shrink one step at a time, the dict gives the jump target directly.

### Code

```
def length_of_longest_substring(s: str) -> int:
    last_seen: dict[str, int] = {}
    left = 0
    longest = 0

    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        longest = max(longest, right - left + 1)

    return longest

if __name__ == "__main__":
    tests = [
        ("abcabcbb", 3),
        ("bbbbbb", 1),
        ("pwwkew", 3),
        ("", 0),
        (" ", 1),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = length_of_longest_substring(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 03. Minimum Size Subarray Sum

03\_minimum\_size\_subarray\_sum.py

### Explanation

Problem: Given a positive-integer array nums and a positive integer target, return the minimal length of a contiguous subarray whose sum is  $\geq$  target. Return 0 if no such subarray exists.

Source : LeetCode 209 - Minimum Size Subarray Sum

Example:

Input: target = 7, nums = [2, 3, 1, 2, 4, 3]

Output: 2 (subarray [4, 3])

Idea: variable-size window, but shrinking instead of growing to find the extreme. Grow the window by moving **right** and adding to a running sum; whenever the sum is already  $\geq$  target, that's a candidate window, so shrink from the left (removing elements, moving **left** forward) for as long as the sum stays  $\geq$  target, recording the window length at every valid shrink step.

### Code

```
def min_subarray_len(target: int, nums: list[int]) -> int:
    left = 0
    total = 0
    min_len = float("inf")

    for right, n in enumerate(nums):
        total += n
        while total >= target:
            min_len = min(min_len, right - left + 1)
            total -= nums[left]
            left += 1

    return 0 if min_len == float("inf") else min_len

if __name__ == "__main__":
    tests = [
        (7, [2, 3, 1, 2, 4, 3], 2),
        (4, [1, 4, 4], 1),
        (11, [1, 1, 1, 1, 1, 1, 1, 1], 0),
        (15, [1, 2, 3, 4, 5], 5),
    ]

    for i, (target, nums, expected) in enumerate(tests, 1):
        got = min_subarray_len(target, nums)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 04. Longest Repeating Character Replacement

04\_longest\_repeating\_character\_replacement.py

### Explanation

Problem: Given a string *s* of uppercase letters and integer *k*, you may replace up to *k* characters in the string with any other uppercase letter. Return the length of the longest substring achievable where every character is the same after doing so.

Source : LeetCode 424 - Longest Repeating Character Replacement

Example:

Input: *s* = "AABABBA", *k* = 1

Output: 4 (replace one 'A' at index 3 -> "AABBBBA" contains "ABBB"? actually "BABB"-> replace to get "AABBBBA", longest run of one letter after 1 replacement is 4)

Idea: variable window tracked with a frequency count of the window's characters. A window of length L needs at most k replacements to become uniform if  $(L - \text{count of its most frequent character}) \leq k$  - i.e. everything that ISN'T the majority character must be replaceable within budget k. Grow **right** always; shrink **left** only when the window violates that budget. **max\_count** is allowed to go stale (never decremented on shrink) because the answer only cares about the best window ever seen, not the exact max\_count for the current window - a well-known simplification for this specific problem.

## Code

```
def character_replacement(s: str, k: int) -> int:
    counts: dict[str, int] = {}
    left = 0
    max_count = 0
    longest = 0

    for right, ch in enumerate(s):
        counts[ch] = counts.get(ch, 0) + 1
        max_count = max(max_count, counts[ch])

        window_len = right - left + 1
        if window_len - max_count > k:
            counts[s[left]] -= 1
            left += 1

        longest = max(longest, right - left + 1)

    return longest

if __name__ == "__main__":
    tests = [
        ("ABAB", 2, 4),
        ("AABABBA", 1, 4),
        ("AAAA", 0, 4),
        ("ABBB", 0, 3),
    ]

    for i, (s, k, expected) in enumerate(tests, 1):
        got = character_replacement(s, k)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 05. Permutation In String

05\_permutation\_in\_string.py

### Explanation

Problem: Given strings s1 and s2, return True if s2 contains a permutation of s1 as a contiguous substring (i.e. some substring of s2 is an anagram of s1).

Source : LeetCode 567 - Permutation in String

Example:

Input: s1 = "ab", s2 = "eidbaooo"

Output: True ("ba" at index 3..4 is a permutation of "ab")

Idea: fixed-size window, size = len(s1). Maintain a running character count of the current window in s2 and compare it to s1's character count. Slide the window one step at a time (add the entering character, remove the one leaving); whenever the two count dicts are exactly equal, a permutation has been found. Dict equality (==) compares keys and values, so this is a single O(1)-ish check per step rather than resorting/recomparing strings.

### Code

```
def check_inclusion(s1: str, s2: str) -> bool:
    if len(s1) > len(s2):
        return False

    need: dict[str, int] = {}
    for ch in s1:
        need[ch] = need.get(ch, 0) + 1

    window: dict[str, int] = {}
    k = len(s1)

    for i, ch in enumerate(s2):
        window[ch] = window.get(ch, 0) + 1

        if i >= k:
            left_ch = s2[i - k]
            window[left_ch] -= 1
            if window[left_ch] == 0:
                del window[left_ch]

        if window == need:
            return True

    return False

if __name__ == "__main__":
    tests = [
        ("ab", "eidbaooo", True),
        ("ab", "eidboaoo", False),
        ("adc", "dcda", True),
        ("hello", "ooolleoooleh", False),
    ]

    for i, (s1, s2, expected) in enumerate(tests, 1):
        got = check_inclusion(s1, s2)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 06. Two Sum Sorted

06\_two\_sum\_sorted.py

### Explanation

Problem: Given a sorted array `numbers` and integer target, return the 0-indexed positions of the two numbers that add up to target. (LeetCode's original returns 1-indexed positions - this version returns 0-indexed to stay consistent with the rest of this repo.)  
Source : LeetCode 167 - Two Sum II - Input Array Is Sorted

Example:

Input: numbers = [2, 7, 11, 15], target = 9

Output: [0, 1]

Idea: because the array is sorted, a converging two-pointer beats the hashmap approach from 02\_hashing/01\_two\_sum on space -  $O(1)$  instead of  $O(n)$ . If the current pair's sum is too small, the only way to increase it is to move `left` right (larger value). If too big, move `right` left. Sortedness is what guarantees this greedy move is safe.

## Code

```
def two_sum_sorted(numbers: list[int], target: int) -> list[int]:
    left, right = 0, len(numbers) - 1

    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left, right]
        elif total < target:
            left += 1
        else:
            right -= 1

    return []

if __name__ == "__main__":
    tests = [
        ([2, 7, 11, 15], 9, [0, 1]),
        ([2, 3, 4], 6, [0, 2]),
        ([-1, 0], -1, [0, 1]),
    ]

    for i, (numbers, target, expected) in enumerate(tests, 1):
        got = two_sum_sorted(numbers, target)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 07. Three Sum

07\_three\_sum.py

### Explanation

Problem: Given an integer array `nums`, return all unique triplets `[a, b, c]` such that  $a + b + c == 0$ . No duplicate triplets in the result.

Source : LeetCode 15 - 3Sum

Example:

Input: [-1, 0, 1, 2, -1, -4]

Output: [[-1, -1, 2], [-1, 0, 1]]

Idea: sort first. Fix one element (`nums[i]`), then the remaining "find two numbers that sum to `-nums[i]`" is exactly the converging two-pointer from `06_two_sum_sorted`, run once per choice of `i`. Sorting also makes duplicate-skipping trivial (identical values end up adjacent, so just compare to the previous element/pointer position). Break early once `nums[i] > 0`, since the array is sorted and three non-negative numbers (with at least one positive) can't sum to zero.

## Code

```
def three_sum(nums: list[int]) -> list[list[int]]:
    nums = sorted(nums)
    n = len(nums)
    result = []

    for i in range(n):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        if nums[i] > 0:
            break

        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1

    return result

def _normalize(triplets: list[list[int]]) -> list[list[int]]:
    return sorted(triplets)

if __name__ == "__main__":
    tests = [
        ([-1, 0, 1, 2, -1, -4], [[-1, -1, 2], [-1, 0, 1]]),
        ([0, 1, 1], []),
        ([0, 0, 0], [[0, 0, 0]]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = three_sum(inp)
        status = "PASS" if _normalize(got) == _normalize(expected) else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 08. Container With Most Water

08\_container\_with\_most\_water.py

### Explanation

Problem: Given an array `height` where `height[i]` is the height of a vertical line at position `i`, find two lines that, together with the x-axis, form a container holding the most water. Return the max area.

Source : LeetCode 11 - Container With Most Water

Example:

Input: [1, 8, 6, 2, 5, 4, 8, 3, 7]

Output: 49 (lines at index 1 (height 8) and index 8 (height 7):

$\text{area} = \min(8, 7) * (8 - 1) = 7 * 7 = 49$ )

Idea: start with the widest possible container (both ends). Area is limited by the shorter of the two lines, so moving the pointer at the *\*taller\** line inward can only keep the height the same or make it worse (still capped by the other, shorter side) while width shrinks - guaranteed loss. Moving the *\*shorter\** line's pointer inward is the only move that could possibly find a taller line and improve the area. That greedy justification is what makes this  $O(n)$  safe (no need to check every pair).

### Code

```
def max_area(height: list[int]) -> int:
    left, right = 0, len(height) - 1
    best = 0

    while left < right:
        h = min(height[left], height[right])
        best = max(best, h * (right - left))

        if height[left] < height[right]:
            left += 1
        else:
            right -= 1

    return best

if __name__ == "__main__":
    tests = [
        ([1, 8, 6, 2, 5, 4, 8, 3, 7], 49),
        ([1, 1], 1),
        ([4, 3, 2, 1, 4], 16),
        ([1, 2, 1], 2),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = max_area(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 09. Sort Colors

## Explanation

Problem: Given an array `nums` with only values 0, 1, 2 (representing red, white, blue), sort it in place in one pass, without using a library sort. (Dutch National Flag problem.)

Source : LeetCode 75 - Sort Colors

Example:

Input: [2, 0, 2, 1, 1, 0]

Output: [0, 0, 1, 1, 2, 2]

Idea: three-pointer partition instead of a two-pointer mirror. `low` and `mid` start at the front, `high` at the back.

- Everything before `low` is known to be 0.
- Everything from `low` to `mid-1` is known to be 1.
- Everything after `high` is known to be 2.
- `mid` is the current element being classified.

A 0 at `mid` swaps to the `low` boundary and both advance (the swapped-in value at `mid` is already known-good, a 1, since region `[low, mid)` was all 1s - safe to move `mid` too). A 2 swaps to the `high` boundary and only `high` shrinks, because the value swapped in from `high` hasn't been classified yet and still needs to be checked.

## Code

```
def sort_colors(nums: list[int]) -> None:
    low, mid, high = 0, 0, len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1

if __name__ == "__main__":
    tests = [
        ([2, 0, 2, 1, 1, 0], [0, 0, 1, 1, 2, 2]),
        ([2, 0, 1], [0, 1, 2]),
        ([0], [0]),
        ([1], [1]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        arr = inp[:]
        sort_colors(arr)
        status = "PASS" if arr == expected else "FAIL"
        print(f"Test {i}: {status} (got {arr}, expected {expected})")
```

## 10. Find The Duplicate Number

10\_find\_the\_duplicate\_number.py

### Explanation

Problem: Given an array `nums` of  $n+1$  integers where every value is in  $[1, n]$ , exactly one value repeats (possibly more than once). Find it without modifying the array and using  $O(1)$  extra space.

Source : LeetCode 287 - Find the Duplicate Number

Example:

Input: [1, 3, 4, 2, 2]

Output: 2

Idea: fast-slow pointers (Floyd's cycle detection) - a preview of the technique this repo will reuse constantly in 04\_linked\_list for cycle detection. Treat the array as a linked list where index  $i$  points to `nums[i]`. Because a value repeats, two different indices point to the same next index, which creates a cycle. Two-phase algorithm:

1. Move slow one step (`nums[slow]`) and fast two steps (`nums[nums[fast]]`) until they meet inside the cycle.
2. Reset one pointer to the start; move both one step at a time - they meet again exactly at the cycle's entrance, which is the duplicate value. (Standard Floyd's algorithm property.)

### Code

```
def find_duplicate(nums: list[int]) -> int:
    slow = nums[0]
    fast = nums[0]

    while True:
        slow = nums[slow]
        fast = nums[nums[fast]]
        if slow == fast:
            break

    slow2 = nums[0]
    while slow2 != slow:
        slow2 = nums[slow2]
        slow = nums[slow]

    return slow

if __name__ == "__main__":
    tests = [
        ([1, 3, 4, 2, 2], 2),
        ([3, 1, 3, 4, 2], 3),
        ([1, 1], 1),
        ([1, 1, 2], 1),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = find_duplicate(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```